`DataContext`. Whatever value `DataContext` was created with, the `Repository` displays the same value.

Listing 9.11 Repository service that depends on an instance of `DataContext`

```
public class Repository
{
    private readonly DataContext _dataContext;        ❶
    public Repository(DataContext dataContext)        ❶
    {                                                 ❶
        _dataContext = dataContext;                   ❶
    }                                                 ❶
    public int RowCount => _dataContext.RowCount;     ❷
}
```

❶ An instance of DataContext is provided using DI.

❷ RowCount returns the same value as the current instance of DataContext.

Finally, you have your endpoint handler, `RowCounts`, which takes a dependency on both `Repository` and on `DataContext` directly. When the minimal API infrastructure creates the arguments needed to call `RowCounts`, the DI container injects an instance of `DataContext` and an instance of `Repository`. To create `Repository`, it must create a second instance of `DataContext`. Over the course of two requests, four instances of `DataContext` will be required, as shown in figure 9.6.